Collisions & Game State Cheat Sheet

CS4040 · Unit 3

> The three rules

1. **End** the game with `game.win()`, `game.lose()` or `game.over()`. Add a number to restart after that many seconds.
2. **Start over right now** with `game.restart()`.
3. **Set up everything for a fresh game in `start()`**: Actor positions *and* your own variables (like `lives`). `start()` runs again every time the game restarts.

Assets

Folder	Holds	File types
`images`	pictures for Actors	`.png`
`sounds`	sound effects	`.wav` or `.ogg`
`music`	background music	`.ogg` or `.mp3`

Asset names follow the same rules as Python variables: all lowercase, no spaces, only letters, numbers and underscores, and they must begin with a letter. Refer to an asset by its name without the extension: `"alien"`, not `"alien.png"`.

Collisions

`colliderect()` is a Boolean expression: `True` if Actor `actor1` has collided with Actor `actor2`, `False` otherwise.

```
if actor1.colliderect(actor2):  # True if actor1 collided with actor2
    game.over()
```

Hitboxes are rectangles around the whole image, including any transparent space.

The `game` object

`game` is an object variable storing information about the game's state.

```
game  # object that stores information about the game's state
game.state  # str: "playing", "won", "lost", "over" or "paused"
game.score  # int: starts at 0. Update it in your code.
game.level  # int: starts at 1. Update it in your code.
game.timer  # float: seconds since game started. Auto-updates.
game.frame_count  # int: frames since game started. Auto-updates.
```

Score, level and timer

```
game.score = 0  # sets the player's score
game.score += 10  # adds points to the score
game.level = 1  # stores the current level number
game.level += 1  # moves to the next level
if game.timer > 5:  # checks if a certain amount of time has passed
f"{game.timer:.2f}"  # timer as a string, rounded to 2 decimal places
f"{int(game.timer)}"  # timer as a string, decimals dropped (whole number)
```

`game.timer` only counts while the game is `"playing"`.

Game states

`game.state` is a string variable that tracks the state of the game. Up to you to implement!

State	Meaning
`"playing"`	Default value. The game is running.
`"won"`	The game is won. Set with `game.win()`.
`"lost"`	The game is lost. Set with `game.lose()`.
`"over"`	The game is over. Set with `game.over()`, used when there is no difference between win/lose.
`"paused"`	The game is paused. Set with `game.pause()`.

You can add your own states! Just set `game.state` equal to any string. You define the value, check it elsewhere in your code, and implement what happens in that state. Other useful states could be `"menu"` or `"credits"`.

Checking the state

```
if game.state == "playing":  # run code only when the game is active
if game.state != "won":  # run code only when the game was not won
```

Ending the game

```
# each one also cancels everything scheduled on the clock
game.win()  # sets game.state to "won"
game.lose()  # sets game.state to "lost"
game.over()  # sets game.state to "over"
```

Adding a number restarts the game after that many seconds:

```
game.win(2.5)  # like game.win(), then restart after 2.5 seconds
game.lose(3)  # like game.lose(), then restart after 3 seconds
game.over(5)  # like game.over(), then restart after 5 seconds
```

These only work while `game.state` is `"playing"`. Once the game has ended, extra calls are ignored, so it's safe to call them from `update()` every frame.

Restarting right now

```
game.restart()  # starts a new game right away, from any state
```

Use this when the player presses a key to play again. It resets `game.state`, `game.score`, `game.level` and `game.timer`, then runs `start()` again.

Pause and resume

While paused, `game.timer` and Sprite animations stop, but `update()` and the clock keep running.

```
game.pause()  # sets game.state to "paused"
game.resume()  # sets game.state back to "playing"
```

Example: game state in a game

`start()` runs automatically when the game begins and again every time it restarts. `draw()` uses `game.state` to decide what to draw. `update()` returns right away unless the game is `"playing"`, then calls `game.lose()` or `game.win()` when the player hits the enemy or the key.

```
def start():  # runs at the beginning and again on every restart
    player.pos = (200, 200)
    enemy.pos = (400, 400)
    key.pos = (450, 200)  # reach the key to win


def draw():
    screen.clear()
    key.draw()
    if game.state != "lost":  # draw player if "playing" or "won"
        player.draw()
    if game.state != "won":  # draw enemy if "playing" or "lost"
        enemy.draw()
    if game.state == "won":
        screen.draw.text("You won!", midtop=(250, 20))
    if game.state == "lost":
        screen.draw.text("Game over", midtop=(250, 20))


def update():
    if game.state != "playing":  # not playing: skip the rest
        return
    player.x += 1
    if player.colliderect(enemy):
        game.lose(5)  # "lost", then restart after 5 seconds
    if player.colliderect(key):
        game.win(3)  # "won", then restart after 3 seconds
```

Heads-up display (HUD)

An example of displaying the score, time (as a whole number), level and a game over message in `draw()`:

```
def draw():
    screen.clear()
    # ... draw the actors and scene here, then the HUD on top
    screen.draw.text(f"Score: {game.score}", midleft=(20, 20))
    screen.draw.text(f"Time: {int(game.timer)}", center=(WIDTH // 2, 20))
    screen.draw.text(f"Level: {game.level}", midright=(WIDTH - 20, 20))
    if game.state == "lost":
        screen.draw.text("Game over!", center=(WIDTH // 2, HEIGHT // 2))
    if game.state == "won":
        screen.draw.text("You won!", center=(WIDTH // 2, HEIGHT // 2))
```